

Laboratorio Semana 8 – Cifrado AES-256, HMAC-SHA256, SSH y Git

V

Nombre: **Victor David Perez Argueta**

Objetivo Aplicar cifrado simétrico para proteger la confidencialidad de un archivo, utilizar HMAC-SHA256 como huella autenticada y trabajar con un repositorio compartido de GitLab mediante SSH.

Herramientas: PowerShell + Git + OpenSSL

Regla de seguridad: Nunca subas la contraseña o clave secreta al repositorio de GitLab.

1. Conexión al repositorio mediante SSH Verifica

Git: `git --version`

Prueba la conexión: `ssh -T git@github.com`

Si es la primera conexión, verifica el fingerprint del servidor antes de aceptarlo. Clona el repositorio compartido: `git clone git@github.com:[tu_repo]_git`

Entra al repositorio y crea tu rama: `cd git checkout -b lab-[nombre_branch]`

2. Crear un archivo personalizado

Crea un archivo cuyo nombre incluya tu nombre y apellido. Cada estudiante debe utilizar contenido personalizado.

Ejemplo: `mensaje_LuisRojas.json` Contenido:

```
{  
  "nombre": "TU NOMBRE",  
  "curso": "Manejo e Implementación de Archivos",  
  "carne": "[tu_carne]",  
  "archivo": "mensaje_[Nombre_apellido].txt"  
}
```

3. Cifrar el archivo con AES-256

Utiliza **OpenSSL** para cifrar tu archivo con **AES-256-CBC**. El archivo resultante tendrá extensión `.enc`

3.1 Verifica OpenSSL

Ejecuta el siguiente comando en **PowerShell**: `openssl version`

Si OpenSSL no está instalado, ejecuta: `winget install --id ShiningLight.OpenSSL.Light --exact` Si instalaste OpenSSL, **reinicia VS Code** antes de continuar.

3.2 Comprueba instalación

`Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

3.3 Cifra tu archivo

Comprueba dónde quedó instalado: `Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

Agrega OpenSSL al PATH de tu usuario: `$opensslPath = "C:\Program Files\OpenSSL-Win64\bin"`

`[Environment]::SetEnvironmentVariable("Path", $env:Path + ";" + $opensslPath, "User")`

Actualiza el PATH de esta terminal inmediatamente: `$env:Path += ";C:\Program Files\OpenSSLWin64\bin"`

3.4 Cifra tu archivo

Ejecuta: `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.json -out .\mensaje_TuNombre.enc`

```
PS C:\Users\perez\Desktop\practica_url_8\url-mia-2026\Semana_8\Lab08> Get-ChildItem "C:\Program Files" -Recurse -Filter openssl.exe -ErrorAction SilentlyContinue

Directorio: C:\Program Files\Common Files\Autodesk Shared\Components\2027\2.0.4

Mode                LastWriteTime         Length Name
----                -
-a----           10/02/2026    03:58         820056 openssl.exe
```

V

Cuando OpenSSL solicite la contraseña, introduce una contraseña que solo tú conozcas. Deberás introducirla dos veces. **NO escribas la contraseña en este laboratorio ni la subas a GitHub.**

3.5 Comprueba que se creó el archivo: Ejecuta: `Get-ChildItem`

Deben aparecer, como mínimo: `mensaje_TuNombre.txt` `mensaje_TuNombre.enc`

3.6 Comprueba que el archivo cifrado no muestra directamente el mensaje:

`Get-Content .\mensaje_TuNombre.enc` ¿Qué observaste?

```
PS C:\Users\perez\Desktop\practica_url_8\url-mia-2026\Semana_8\Lab08> Get-Content .\mensaje_Victor_Perez.enc
Salted__Yâ÷éræí™JðùzðuWÊ/æ@Pé+wîbL,,µçðáLn4ÚUi,±-°w
                                ùÚÝøf?Ê-e5P
                                f³ëWú"2'gk*ŷ@yGô%ö
zÚHduõRo€...@UdòòYw"il÷ù9Â#QCE,,{õ€b!xµd•û@xgö°çİ'~ÖG~wAçúÿ-î;K8yÚ÷ë2
PS C:\Users\perez\Desktop\practica_url_8\url-mia-2026\Semana_8\Lab08>
```

4. Descifrar y recuperar el archivo

Utiliza la misma contraseña para recuperar el contenido original.

`openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.enc -out .\mensaje_TuNombre_recuperado.json`

Comprueba el contenido recuperado: `Get-Content .\mensaje_TuNombre_recuperado.json`

```

PS C:\Users\perez\Desktop\practica_url_8\url-mia-2026\Semana_8\Lab08> Get-Content .\mensaje_Victor_Perez_recuperado.json
{
  "nombre": "Victor Perez",
  "curso": "Manejo e Implementaci3n de Archivos",
  "carne": "1074325",
  "archivo": "mensaje_Victor_Perez.txt"
}

PS C:\Users\perez\Desktop\practica_url_8\url-mia-2026\Semana_8\Lab08>

```

5. Generar una huella autenticada con HMAC-SHA256

Ahora generarás una huella que depende del archivo y de una clave secreta. Esto permite comprobar integridad y autenticidad cuando quien verifica también conoce la clave.

archivo + clave secreta → HMAC-SHA256 → huella

corre: `openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_LuisRojas.json` ¿Cuál es el HMAC-SHA256 obtenido?

HMAC-SHA2-256(.\mensaje_Victor_Perez.json)=

9df9ea1f1cf8343515ee688f20cd9690bcbb8f43fbf0420cac404709ec5a0be8

6. Hash vs. cifrado vs. HMAC

Mecanismo	Objetivo	Clave	Ejemplo
SHA-256	Integridad	No	Huella
AES-256 (encrypt)	Confidencialidad	Sí	Archivo .enc
HMAC-SHA256	Integridad + autenticidad	Sí	Huella autenticada
SSH	Comunicación/autenticación segura	Sí	GitLab

Pregunta 1: Si alguien obtiene tu archivo .enc pero no conoce la contraseña, ¿qué propiedad estás protegiendo?

R// Se está protegiendo la **confidencialidad**, ya que el contenido del archivo no puede ser leído fácilmente sin conocer la contraseña.

Pregunta 2: ¿Por qué no debes subir la contraseña a GitLab?

R// Porque otras personas podrían tener acceso a ella y utilizarla para descifrar el archivo, haciendo que la protección deje de ser efectiva.

Pregunta 3: ¿Qué diferencia principal existe entre SHA-256 y HMAC-SHA256?

R// SHA-256 genera una huella a partir del contenido del archivo, mientras que HMAC-SHA256 también utiliza una clave secreta, lo que permite verificar tanto que el archivo no fue modificado como que proviene de alguien que conoce esa clave.

7. Entrega

Dentro del repositorio de GitHub, crea la carpeta semana-8/lab8/ y coloca en ella un PDF con la solución del laboratorio, incluyendo las respuestas, el HMAC-SHA256 obtenido y pantallazos como evidencia de los comandos y procedimientos realizados. También incluir el archivo .json creado y el archivo .enc generado durante el cifrado. Enviar en la plataforma el enlace de la carpeta lab8 como evidencia de la entrega.